DEVELOPMENT REPORT & PROMPT ENGINEERING OVERVIEW: NEON SNAKE PRO

Project Title: Neon Snake Pro
Version: 1.2.0
Lead Developer: Salman Ahmed
Platform: React, Vite, HTML5 Canvas, Tailwind CSS
Deployment Link: https://ais-pre-ez35bujfvwwwcoqnrnsg3i-973927816057.asia-southeast1.run.app

---

1. Prompt Iterations & Evolution

During the development of **Neon Snake Pro**, we utilized an iterative prompting process to transition the application from a raw prototype into a polished, responsive, and robust HTML5 game.

Phase 1: Blueprint & Baseline Mechanics

- **Initial Prompt:** *"Create a React-based Neon Snake game with modern glassmorphism styling, clean animations, local achievements, and offline-ready statistic tracking counters."*
- **Outcome:** Established the core game loop, basic canvas grid, and the initial stats/achievements views.

Phase 2: Bug Mitigation & Game State Syncing

- **Refinement Prompt:** *"Fix the React rendering warning about state updates during render state cycles. Defer parent state tracking checks using setTimeout macros and handle nested callbacks safely."*
- **Outcome:** Dissolved React render lifecycle warnings and decoupled side-effect computations.

Phase 3: Layout Polish & Container Fitting

- **Polish Prompt:** *"Ensure full vertical scrollability for Stats, Achievements, and Settings views so that no buttons are cut off. Maintain a single-container layout but transition parent heights to fit fluid mobile viewports."*
- **Outcome:** Improved CSS layouts on small screens, preventing clipping of interactive dashboards.

Phase 4: Control Mechanics & Gameplay Bug Fixes

- **Final Alignment Prompt:** *"Ensure the pause button halts the loop. Make sure the direction interval is skipped while the state isPaused is active, and update the game loop's dependency array to track state changes correctly."*
- **Outcome:** Resolved the loop leak where the game kept running underneath the pause window.

---

2. What Worked Well

- **Direct WebGL/Canvas Rendering:** Keeping snake coordinates in a raw Cartesian plane and rendering on a single `<canvas>` context minimized React's render overhead, yielding a persistent 60fps frame rate.
- **Decoupled Local Storage:** Storing achievements and stats locally in JSON-serialized format worked reliably and persisted smoothly across page reloads.
- **Post-Render Event Deferrals:** Utilizing `setTimeout(..., 0)` to push parent-level metrics updates out of the active component render cycle resolved critical React architecture conflicts.

---

3. What Did Not Work

- **State Updates in the Render Phase:** Directly firing score updates, statistics commits, and achievement checks inside game updates caused unexpected React re-render crashes.
- **Strict Height Viewports:** Using rigid Tailwind utility classes (like `h-screen` and `overflow-hidden`) cut off vital UI views and stats boards on mobile screens, requiring a shift to standard responsive scrolling layout.
- **Missing Hook Dependency Trackers:** Excluding the `isPaused` boolean from the main `useEffect` game loop dependency list allowed old interval cycles to continue running, bypassing the pause menu entirely.

---

4. How Prompts Were Improved Over Time

- **Shifting from Feature Requests to Structural Refinement:** Early prompts focused purely on functional features (like adding golden apples). Later prompts shifted to specific structural bugs, UI density optimizations, and precise React hook configurations.
- **Adding Technical Constraints:** Prompts evolved from general descriptions (*"make it responsive"*) to technical requirements (*"adjust card padding on small breakpoints, decrease typography display scales, and set h-auto containers"*), which produced cleaner, targeted results.

---

5. Challenges Faced

- **Mobile Viewport Integrity:** Scaling down the game canvas, high-impact buttons, and headers without breaking single-line branding titles ("NEON SNAKE PRO") required strict media-query tuning and responsive layout adjustments.

- **Threading the Pause Toggle:** Preventing the background clock and game loop thread from advancing while the UI overlay showed a paused state required rewriting the React hook interval dependencies.
- **Unique Database Key Collisions:** Managing multiple dynamic notifications for unlocked accomplishments without producing duplicate element key warnings required sanitizing the array inputs and checking for pre-existing keys before rendering.